
Memoria de Proyecto

Proyecto Intermodular

**Sistema de Gestión de Colas y
Turnos en Tiempo Real**



Sergio Cano Marquez
2-CFGS DAM

1. Resumen y Uso del Proyecto	3
2. Guía de Ejecución	3
3. Relación de Evidencias	4
• Evidencia 1 (Acceso a datos)	4
• Evidencia 2 (Sistemas ERP-CRM)	5
• Evidencia 3 (Nube AWS)	7
• Evidencia 4 (Sockets)	9
• Evidencia 5 (App Móvil)	11
• Evidencia 6 (Docker)	13
• Evidencia 7 (GitHub)	14
4. Reflexión Final	15

1. Resumen y Uso del Proyecto

El Sistema de Gestión de Colas y Turnos en Tiempo Real para la Clínica Azucena es una plataforma cliente-servidor diseñada para optimizar el flujo de pacientes, mejorar la experiencia del usuario y evitar las aglomeraciones físicas.

El flujo de uso es el siguiente: el paciente solicita su turno a través de una aplicación móvil intuitiva desarrollada en React Native, donde recibe un ticket virtual y visualiza una estimación de su tiempo de espera. Por otro lado, la clínica cuenta con un servidor backend concurrente en Java que centraliza la lógica y mantiene conexiones activas. Cuando el operario o médico llama al siguiente paciente, el servidor sincroniza inmediatamente la información, actualizando en tiempo real tanto la aplicación del usuario como las pantallas públicas de la sala de espera, las cuales están desarrolladas en HTML/CSS/JS .

2. Guía de Ejecución

Los pasos para ejecutar el entorno al completo son los siguientes:

1. Levantar BBDD. En la raíz del proyecto, ejecutar con doble click el archivo [ejecutar.bat](#), de manera que arranquen automáticamente los contenedores del docker-compose.yml.
2. Iniciar Servidor Java. Con los contenedores activos, abrir el backend del proyecto en la carpeta [src](#) y ejecutar la clase principal [Main.java](#). Esto iniciará el servidor de socket y se conectará a las bases de datos. Una vez arrancado, el panel del operario queda activo en la terminal. Pulsando 's' se llama al siguiente turno. Si el servidor se reinició entre sesiones y la base de datos tiene tickets de días anteriores, escribir 'sx' donde x es el número del último turno en pantalla sincroniza la pantalla con los tickets ya emitidos.
3. Iniciar el Frontend. Una vez está el backend activo, falta abrir el panel web, el cual se encuentra en la carpeta [panel-web](#) en la raíz del proyecto, y dentro de esta, el archivo [index.html](#). Por último, abrir la aplicación móvil, accediendo a la carpeta [TurnosApp](#), abrir terminal en esta y ejecutar el comando [npx expo run:android](#)

3. Relación de Evidencias

- **Evidencia 1 (Acceso a datos)**

Se requería una base de datos para almacenar la información de los turnos de los pacientes en MySQL y conectarlo con el servidor Java.

Primero creé el documento "turnos.sql" que contiene la estructura de la tabla.

Monté el entorno de base de datos usando Docker pero hay otras formas de hacerlo. Docker, sin embargo, es la más fácil con diferencia. También hice la lógica dentro de BBDD.java, que se usa para conectar con la base de datos. Después usé otra clase, Credenciales.java, que se encarga de coger las contraseñas y puertos desde un archivo properties para arrancar. La ruta para acceder a turnos.sql desde la raíz es: src/turnos.sql.

En BBDD, tengo el método registrarNuevoTurno. Este método recoge una clase de tipo MensajeRed y hace una conexión con la base de datos. BBDD inserta los datos y devuelve el número del ticket que ha generado, o si falla devuelve un "NO_RECIBIDO" para debug.

Las clases se encuentran en la carpeta src.

```
public String registrarNuevoTurno (MensajeRed mensaje) {  
  
    Credenciales cre = new Credenciales();  
  
    try {  
  
        cre.inicializar();  
  
    } catch (Exception e) {  
  
        logger.log(Level.SEVERE, "Exception in main:  
"+e.getMessage()+"", e);  
  
    }  
  
    String user = cre.getUser_DATABASE();  
  
    String password = cre.getPASS_DATABASE();  
  
    String url = cre.getUrl_DATABASE();  
  
    try (Connection conexion = DriverManager.getConnection(url, user,  
password)) {  
  
        Statement statement = conexion.createStatement();
```

```

        {
            String script = "INSERT INTO turnos (rol, accion) VALUES
(?, ?)";

            try (PreparedStatement ps =
conexion.prepareStatement(script, Statement.RETURN_GENERATED_KEYS)) {

                ps.setString(1, mensaje.getRol());

                ps.setString(2, mensaje.getAccion());

                ps.executeUpdate();

                System.out.println("datos guardados correctamente.");

                ResultSet rs = ps.getGeneratedKeys();

                rs.next();

                return "T-" + rs.getInt(1);

            }

        }

    } catch (Exception e) {

        logger.log(Level.SEVERE, "Exception in thread
"+Thread.currentThread().getName()+": "+e.getMessage()+".", e);

    }

    return "NO_RECIBIDO";

}

}

```

- **Evidencia 2 (Sistemas ERP-CRM)**

El sistema se integra con el CRM de la clínica en Odoo para sincronizar los datos de los clientes y el registro de sus atenciones finalizadas dentro del módulo CRM.

Para realizarlo, usé el protocolo XML-RPC en java, que es la forma de hablar con la api de odoo mediante la clase SincronizadorOdoo.java, que contiene los datos de acceso y apunta a la BD de Odoo. En el método enviarCliente, se recibe el número de ticket y el nombre del paciente, hace una conexión para autenticarse y si todo va bien, hace una petición para crear un nuevo cliente o contacto en Odoo(res.partner) con el nombre y el número de turno.

```
Object uidObj = client.execute("authenticate", Arrays.asList(
    BD_ODOO, USUARIO, PASSWORD, Collections.emptyMap()
));

if (uidObj instanceof Boolean) {
    System.out.println("cuidado: el usuario o la contraseña
de odoo son incorrectos");
    return;
}

int uid = (Integer) uidObj;
config.setServerURL(new URL(URL_ODOO + "/xmlrpc/2/object"));
Map<String, Object> datosCliente = new HashMap<>();
datosCliente.put("name", nombreReal + " (" + nombreTicket +
");");

Object idPartnerObj = client.execute("execute_kw",
Arrays.asList(
    BD_ODOO, uid, PASSWORD,
    "res.partner", "create",
    Collections.singletonList(datosCliente)
));

int idPartner = (Integer) idPartnerObj;
Map<String, Object> datosCita = new HashMap<>();
datosCita.put("name", "Turno " + nombreTicket);
```

```

        datosCita.put("partner_id", idPartner);

        datosCita.put("description", "el paciente ha completado su
turno");

        client.execute("execute_kw", Arrays.asList(

            BD_ODOO, uid, PASSWORD,

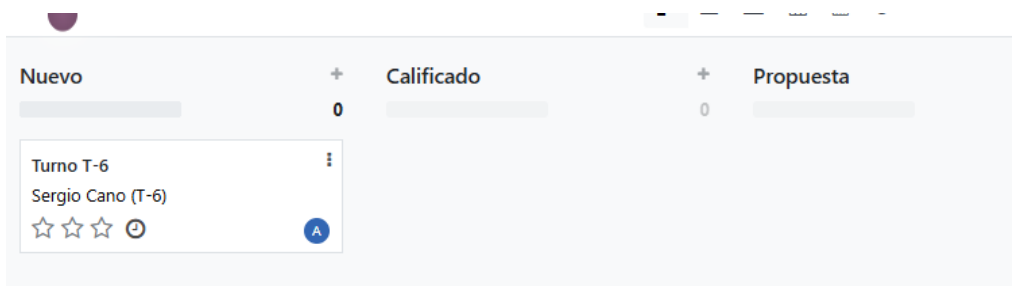
            "crm.lead", "create",

            Collections.singletonList(datosCita)));

```



Inmediatamente después, hace otra petición para crear registro en el crm (crm.lead) vinculado al cliente recién creado para que quede constancia de su turno.

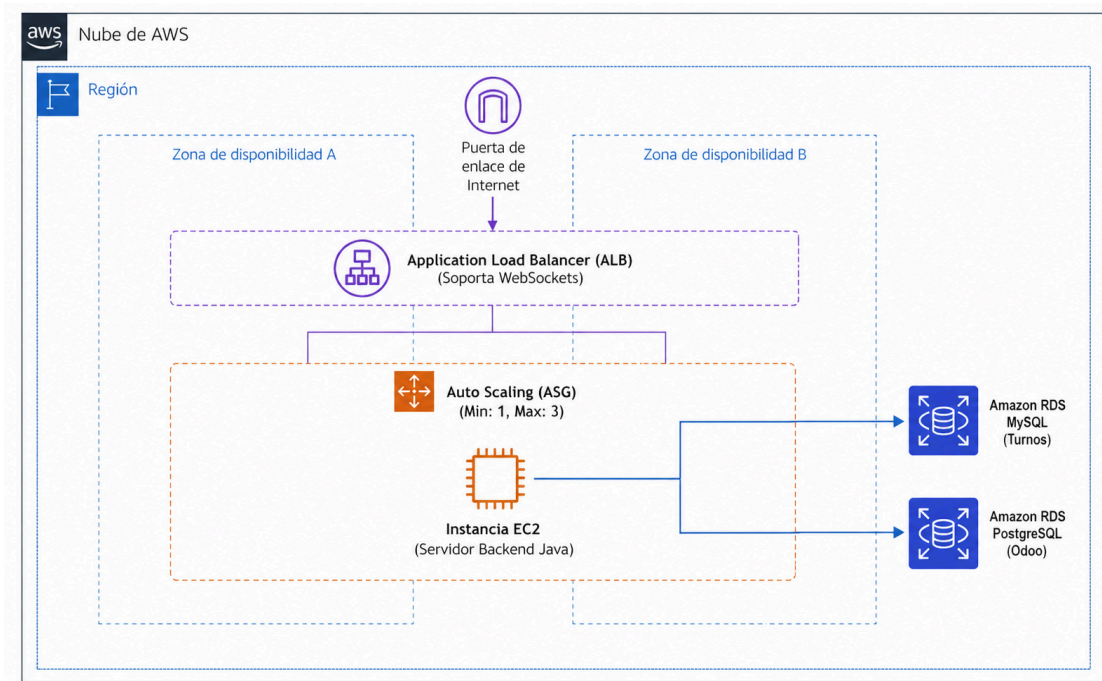


La ruta de este archivo está en src, y se le llama desde GestorCliente.java en el momento que el servidor recibe la petición de pedir turno desde el móvil.

- **Evidencia 3 (Nube AWS)**

Para soportar los picos de concurrencia y garantizar el funcionamiento del sistema en todo momento, es necesario diseñar una arquitectura en AWS.

Quiero aclarar de entrada que para el alcance real de la Clínica Azucena, al ser un negocio pequeño y local, esta solución en la nube no es necesaria y es exagerada, con ejecutarlo en un servidor local es suficiente para el posible volumen de pacientes, pero en el hipotético caso de que fuera necesario cumplir con los requisitos que se piden, esta es la estructura en la nube que yo desplegaría.



ALB: Funciona como la puerta de entrada única al sistema. Su trabajo es recibir todas las peticiones de los móviles y el panel web y repartirlas entre los servidores disponibles. He elegido un ALB porque es capaz de gestionar conexiones persistentes, asegurando que si un servidor se cae, el tráfico se redirija automáticamente a otro que esté sano sin que el usuario pierda su turno.

EC2 y grupos de autoescalado: He configurado un grupo de autoescalado para que, si de repente hay un pico de gente pidiendo turnos y la CPU de los servidores sube, AWS levante automáticamente nuevas máquinas para absorber el tráfico. Una vez que el pico pase, las máquinas sobrantes se apagan solas para ahorrar costes. Esto garantiza que la app nunca vaya lenta, sin importar cuánta gente haya conectada.

RDS MySQL: En lugar de instalar MySQL dentro de un servidor normal, he optado por un servicio gestionado de bases de datos. Esto permite que la base de datos sea independiente de los servidores de aplicaciones. RDS se encarga solo de que los datos de los turnos estén seguros, haciendo copias de seguridad automáticas y permitiendo escalar el almacenamiento de forma aislada.

Utilizando la calculadora oficial de AWS y introduciendo los servicios anteriores el coste mensual de esta infraestructura sería de 93,87 USD.

My Estimate

Editar

Resumen de la estimación

Información

Costo inicial

0,00 USD

Costo mensual

93,87 USD

Costo total de 12 months

1126,44 USD

Incluye el costo inicial

My Estimate

Duplicar

Eliminar

Q

Buscar recursos

☐	Nombr...	Estado	Costo i...	Costo ...	Descrip...	Región	Resumen de la configuración
☐	Amazon EC2	-	0,00 USD	16,64 USD	-	Europa (E...	Tenencia (Instancias compartidas), Sistema operativo
☐	Elastic Lo...	-	0,00 USD	18,48 USD	-	Europa (E...	Número de balanceadores de carga de aplicaciones
☐	Amazon R...	-	0,00 USD	58,75 USD	-	Europa (E...	Cantidad de almacenamiento (20 GB), Almacenamie

Enlace a la estimación:

<https://calculator.aws/#/estimate?id=2eb53fe921fb450a18b90d49994190947ffe4665>

- **Evidencia 4 (Sockets)**

El corazón de este proyecto es el servidor de sockets TCP concurrente en Java, que mantiene conectadas las aplicaciones móviles de los pacientes y emite eventos de red en tiempo real.

Para esto, creé la clase Main.java, que levanta un ServerSocket en el puerto indicado por el archivo properties. En Main hay un bucle infinito que acepta las conexiones entrantes. Cada vez que un móvil se conecta, el servidor crea un hilo independiente usando la clase GestorCliente.java. De esta manera, el servidor no se bloquea esperando a un usuario y puede atender a decenas de dispositivos a la vez de forma concurrente.

```
try (ServerSocket serverSocket = new ServerSocket(puertoTCP)) {  
    System.out.println("servidor escuchando en el puerto " +  
puertoTCP);  
  
    while (true) {  
  
        Socket socketCliente = serverSocket.accept();  
  
        System.out.println("nuevo cliente conectado: " +  
socketCliente.getInetAddress());  
  
        Thread hilo = new Thread(new GestorCliente(socketCliente));  
  
        hilo.start();  
  
    }  
  
    } catch (IOException e) {  
  
        System.out.println("error al arrancar el servidor: " +  
e.getMessage());  
  
        logger.log(Level.SEVERE, "exception en main: " + e.getMessage() +  
".", e);  
  
    }  
}
```

La clase GestorCliente.java implementa Runnable para ejecutarse en ese hilo propio y se encarga de leer constantemente los mensajes de ese socket en concreto. Hago uso de la librería Gson para parsear el texto JSON que llega y convertirlo en un objeto de mi clase MensajeRed.

Si la acción que recibe es "PEDIR_TURNO", llama a la BBDD para registrarlo, extrae el número de ticket generado, lo sincroniza con el CRM Odoo y le responde únicamente a ese cliente enviándole su ticket con la orden "TURNO_ASIGNADO".

```
MensajeRed mensaje = gson.fromJson(nuevo, MensajeRed.class);

if (mensaje != null && mensaje.getAccion() != null) {

    if (mensaje.getAccion().equals("PEDIR_TURNO")) {

        System.out.println("el paciente " + mensaje.getNombre() + " ha  
pedido turno");

        BBDD bd = new BBDD();

        String nuevoTicket = bd.registrarNuevoTurno(mensaje);

        SincronizadorOdoo.enviarCliente(nuevoTicket, mensaje.getNombre());

        MensajeRed respuesta = new MensajeRed("TURNO_ASIGNADO",  
"SERVIDOR", nuevoTicket, "");

        String jsonRespuesta = gson.toJson(respuesta);

        enviarMensaje(jsonRespuesta);
```

Finalmente, para que el sistema tenga capacidad de emitir eventos globales en tiempo real, implementé un orquestador. La clase GestorConexiones.java mantiene una lista en memoria (ArrayList) con todos los clientes que están activos en ese momento. Cuando el operario avanza la cola, se llama al método enviarATodos, el cual recorre la lista completa y hace un broadcast mandando la orden de "ACTUALIZAR_PANTALLA" a todos los móviles conectados a la vez, para que reaccionen y piten si es su turno.

```
private static final List<GestorCliente> clientesActivos = new  
ArrayList<>();

public static synchronized void agregarCliente(GestorCliente c) {

    clientesActivos.add(c);

}

public static synchronized void eliminarCliente(GestorCliente c) {

    clientesActivos.remove(c);

}

public static synchronized void enviarATodos(String mensaje) {

    for (GestorCliente c : clientesActivos) {

        c.enviarMensaje(mensaje);}}
```

- **Evidencia 5 (App Móvil)**

El frontend de la aplicación móvil de los pacientes se ha desarrollado en React Native con la herramienta de Expo, la cual dispone de varias librerías que han facilitado mucho el desarrollo de la app. Para el diseño utilicé clases de Tailwind y para gestionar el estado global la librería Zustand.

El funcionamiento de la aplicación se basa en solicitar turno y visualizar el ticket asignado, calculando una estimación del tiempo de espera mientras se sincroniza en tiempo real con el servidor.

Primeramente, para la interfaz principal, he creado una vista en la que el usuario introduce su nombre. Una vez el servidor le asigna un ticket, la interfaz muestra el número y calcula el tiempo de espera mediante la función calcularMinutos. Estos métodos se encuentran en app/index.tsx.

```
const calcularMinutos = () => {

  if (!tieneTurno || turnoActual === 'T-0' || miTurno === 'Sin turno')
    return 0;

  try {

    const actual = parseInt(turnoActual.replace('T-', ''));

    const mio = parseInt(miTurno.replace('T-', ''));

    const diferencia = mio - actual;

    if (diferencia <= 0) return 0;

    return diferencia * 10;

  } catch (e) {

    return 0;

  }

};

<TextInput

  placeholder="escribe tu nombre completo"

  value={nombre}

  onChangeText={setNombre}

  className="bg-white p-5 rounded-2xl mb-4 border
border-emerald-200 text-lg shadow-sm text-gray-800"

/>
```

```

<ThemedButton

  label="PEDIR CITA"

  className="w-full shadow-lg bg-emerald-700"

  onPress={() => solicitarTurno(nombre)}

/>

```

El componente que completa el flujo es el encargado de enviar y escuchar los datos del servidor Java, con una comunicación persistente mediante socket TCP usando la librería react-native-tcp-socket.

Este módulo establece la conexión con la IP y el puerto del servidor y envía los datos en JSON con la acción requerida y el nombre del paciente. Se encuentra en hooks/useServidor.tsx.

```

const solicitarTurno = (nombrePaciente: string) => {

  if (clienteRef.current) {

    const datos = { accion: "PEDIR_TURN0", nombre: nombrePaciente };

    const jsonEnviar = JSON.stringify(datos);

    console.log("enviando por tcp...", jsonEnviar);

    clienteRef.current.write(jsonEnviar + '\n');
  }
}

```

Para la escucha activa del servidor, en el mismo archivo se realiza la conexión con el evento on('data'). Esto permite que cuando el servidor manda un aviso de que el turno avanza, la app móvil intercepta el contenido JSON y actualiza las variables globales en pantalla sin que el usuario tenga que recargar.

```

clienteRef.current.on('data', (data: any) => {
  const respuesta = data.toString().trim();
  console.log("servidor responde:", respuesta);
  try {
    const datosJson = JSON.parse(respuesta);
    onActualizacion(datosJson);
  } catch (e) {
    console.log("no es json", respuesta);
  }
})

```

- **Evidencia 6 (Docker)**

Para que el proyecto se pueda ejecutar de forma simple y en cualquier ordenador he empleado dockers para la estructura del proyecto.

Con el archivo docker-compose.yml en la raíz del proyecto puedo levantar tres contenedores en la misma red, la base de datos de registro de turnos, la base de datos de Odoos y el propio entorno de Odoos 16 para el crm.

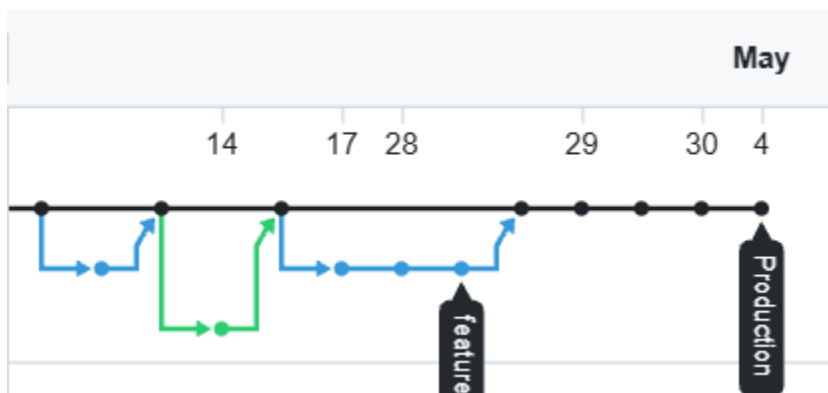
Para hacerlo aún más sencillo, he creado un ejecutable llamado ejecutar.bat que deja los contenedores docker en funcionamiento.

- **Evidencia 7 (GitHub)**

Para el control de versiones y organización del trabajo, he utilizado el siguiente repositorio de github: <https://github.com/sercanmar/TurnosClinicaAzucena>

Para organizar el código, he usado una estructura basada en una rama principal llamada Production y ramas secundarias, feature, para las partes principales del proyecto como el frontend o la base de datos.

He aquí una captura del historial de mi repositorio en GitHub, donde se ve el flujo de las ramas secundarias fusionándose mediante merges a la rama principal a lo largo de los días de desarrollo:



4. Reflexión Final

Este proyecto ha sido un reto bastante exigente, pero a la vez una de las experiencias con las que más he aprendido. Para organizarme el trabajo, fui atacando el proyecto por capas, primero monté la infraestructura con Docker y la base de datos, luego programé el servidor Java con los sockets, y finalmente desarrollé el frontend.

Sin duda la mayor dificultad fue la sincronización de datos con Odoo, porque nunca lo había puesto en práctica así que tuve que investigar cómo funcionaba el protocolo XML-RPC, pelearme con la conexión a la base de datos y hacer muchas pruebas.

También otro dolor de cabeza fue la comunicación TCP, que pese a ya haberla usado en mi anterior proyecto, dio muchos problemas porque no servía directamente para la pantalla web de la sala de espera. Por lo visto los navegadores web con HTML y JS no se comunican de forma nativa mediante Sockets TCP como lo hace una app móvil, entonces, para solucionarlo tuve que hacer un segundo servidor por HTTP. De manera que la web puede hacer peticiones cada 5 segundos y actualizar el turno sin colgarse.

Pero a pesar de las dificultades, he intentado cumplir todas las evidencias lo mejor que he podido, incluyendo todas las tecnologías necesarias.